CODE:

```
# Map Coloring using Backtracking (no external libraries)

regions = ["WA", "NT", "SA", "Q", "NSW", "V", "T"]
colors = ["Red", "Green", "Blue"]

# Define adjacency (neighbors)
neighbors = {
    "WA": ["NT", "SA"],
    "NT": ["WA", "SA", "Q"],
    "SA": ["WA", "NT", "Q", "NSW", "V"],
    "Q": ["NT", "SA", "NSW"],
    "NSW": ["SA", "Q", "V"],
    "V": ["SA", "NSW"],
    "T": []  # Tasmania has no neighbors
}

def is_valid(region, color, assignment):
    """Check if assigning this color is valid given neighbors."""
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

def backtrack(assignment):
    """Recursive backtracking CSP solver."""
    if len(assignment) == len(regions):
        return assignment
```

```
    # Pick next unassigned region
    for region in regions:
        if region not in assignment:
            for color in colors:
                if is_valid(region, color, assignment):
                    assignment[region] = color
                    result = backtrack(assignment)
                    if result:
                        return result
                    assignment.pop(region)
            return None

solution = backtrack({})
print("Solution:", solution)
```

OUTPUT:

